FastAPI & Backend Development Interview Knowledge

API design, validation, security, database sessions, testing and production operations

Purpose
HTTP/REST, FastAPI, Pydantic, dependency injection, authentication, CORS, files, testing and production debugging.

Audience: Final-year engineering students and fresher candidates targeting Python, software development, ML, AI engineering and GenAI roles.

RAG design: Content is explicit and retrieval-friendly so chunks can stand alone. Source identifiers, definitions, examples, troubleshooting and interview reasoning are intentionally separated.

Table of Contents

1. 1. HTTP and REST fundamentals
2. 2. FastAPI routes and Pydantic
3. 3. Dependency injection and DB sessions
4. 4. Backend architecture
5. 5. Authentication, JWT and authorization
6. 6. CORS, environment variables and secrets
7. 7. File uploads, exceptions and logging
8. 8. Testing and documentation

1. HTTP and REST fundamentals

HTTP uses request/response semantics. GET reads, POST commonly creates or triggers, PUT replaces, PATCH partially updates and DELETE removes. Idempotency describes whether repeating the intended operation has the same effect; GET/PUT/DELETE are designed to be idempotent, while POST is not assumed to be.

Status codes let clients classify results: 2xx success, 4xx request/client problems, 5xx server failures. Consistent status semantics make frontend error handling easier.

Code	Meaning
200	OK
201	Created
204	Success without body
400	Bad request
401	Authentication required/invalid
403	Authenticated but forbidden
404	Not found
422	Validation/input error in common FastAPI flows
500	Unexpected server error

2. FastAPI routes and Pydantic

FastAPI uses Python type annotations and Pydantic models to validate and serialize API data. Request schemas should describe the public contract rather than exposing database models directly.

Separate input and output models when fields have different security or lifecycle requirements. Never return plaintext passwords or secret tokens merely because they exist in a database object.

```
from fastapi import FastAPI
from pydantic import BaseModel, Field
app = FastAPI()
class UserCreate(BaseModel):
    email: str
    age: int = Field(ge=18)
class UserOut(BaseModel):
    id: int
    email: str
@app.post("/users", response_model=UserOut, status_code=201)
async def create_user(user: UserCreate):
    return {"id": 1, "email": user.email}
```

3. Dependency injection and DB sessions

FastAPI dependency injection lets endpoints declare reusable requirements such as database sessions, authentication checks or shared parameters. FastAPI resolves dependencies and injects their results.

A database session should have a defined lifecycle. A common pattern is to create a session in a dependency, yield it to the request, and close it afterward. Exact SQLAlchemy setup depends on the installed version and sync/async architecture.

```
from typing import Annotated
from fastapi import Depends

def get_db():
    db = make_session()
    try:
        yield db
    finally:
        db.close()

DB = Annotated[Session, Depends(get_db)]
```

4. Backend architecture

A practical separation is router/controller for HTTP, schema for validation/serialization, service for business rules, repository for data access and infrastructure for external clients/configuration.

Layers are tools, not rules. For a small project, extra abstraction can create noise. For a larger codebase, clear boundaries improve testability and make dependencies replaceable.

Layer	Responsibility
Router	HTTP path/method/auth/response
Schema	Input/output contract
Service	Business logic/orchestration
Repository	Database access
Infrastructure	External clients/configuration

5. Authentication, JWT and authorization

Authentication asks who the user is; authorization asks what they may do. Passwords must be hashed with a secure password-hashing library rather than stored in plaintext.

JWTs are signed tokens, not encrypted containers. Validate signature, expiry and relevant claims. Do not put secrets in JWT payloads. Authorization checks should still verify roles, scopes or resource ownership.

```
from datetime import datetime, timedelta, timezone
import jwt

def create_token(subject, secret):
    now = datetime.now(timezone.utc)
    return jwt.encode(
        {"sub": subject, "exp": now + timedelta(minutes=30)},
        secret, algorithm="HS256"
    )
```

6. CORS, environment variables and secrets

CORS controls which browser origins are allowed to make cross-origin requests. It is not authentication and does not prevent arbitrary non-browser clients from calling an API.

Keep database URLs, API keys and signing secrets outside source code. Environment variables are common in deployment; use platform secret stores where available.

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://frontend.example.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

7. File uploads, exceptions and logging

Validate uploaded file type, size and naming; do not trust client filenames as safe filesystem paths.

Use HTTPException or custom handlers for expected API errors. Log structured context such as request IDs, but never log passwords, bearer tokens or database credentials.

```
from fastapi import HTTPException, UploadFile
@app.post("/documents")
async def upload(file: UploadFile):
    if file.content_type != "application/pdf":
        raise HTTPException(status_code=415, detail="PDF required")
    return {"filename": file.filename}
```

8. Testing and documentation

Unit tests isolate functions/services and should be fast. Integration tests exercise boundaries such as a real database or external service. Test validation, authentication failures, authorization and transaction behavior—not only happy paths.

FastAPI can generate OpenAPI documentation from route declarations, schemas and dependencies. Interactive docs are useful for debugging, but contract tests are still valuable.

```
from fastapi.testclient import TestClient
client = TestClient(app)
def test_create_user():
    r = client.post("/users", json={"email":"test@example.com", "age":21})
    assert r.status_code == 201
```

Scenario: frontend gets a CORS error

Situation: Check the exact browser origin, scheme and port, then inspect the response and preflight request. Confirm the backend is using the intended CORS configuration.

Reasoning: Do not permanently allow every origin just to hide the symptom.

Scenario: API works locally but not after deployment

Situation: Check start command, bind host/port, environment variables, database URL, migrations, external-service credentials and logs. Managed platforms often require listening on a provided port.

Reasoning: Read the startup logs before changing application logic.

Scenario: requests hang under load

Situation: Inspect blocking work inside async routes, database pool limits, external API timeouts and CPU/memory use. `async def` does not make a blocking library non-blocking.

Reasoning: Adding async keywords without identifying the blocking operation.

Interview question

Authentication versus authorization?

Strong answer: Authentication verifies identity. Authorization determines what an authenticated identity can access or modify.

Follow-up: Where should resource ownership be checked?

Interview question

Why use Pydantic schemas?

Strong answer: They define an explicit API contract, validate input and control output serialization, reducing accidental data exposure.

Follow-up: Why separate input/output models?

Interview question

What is FastAPI dependency injection?

Strong answer: The endpoint declares what it needs, such as a DB session or current user, and FastAPI resolves the dependency. This reduces duplication and supports testing and security checks.

Follow-up: How does `Annotated` fit?

Interview question

Why is CORS not security authentication?

Strong answer: CORS is a browser enforcement mechanism. Other clients can call the API without browser restrictions. Authentication must protect the API independently.

Follow-up: What is a preflight request?

Interview question

How should external API failures be handled?

Strong answer: Set timeouts, classify retryable errors, use bounded retries/backoff where appropriate, return controlled failures and log diagnostic context without secrets.

Follow-up: What is a retry storm?

Interview question

Why hash passwords?

Strong answer: Hashing prevents the application from needing to store recoverable plaintext passwords. A secure password-hashing algorithm is intentionally expensive and uses a salt, making offline guessing harder.

Follow-up: Why is encryption different?

Revision checklist

- Explain HTTP methods/status codes.
- Design Pydantic request/response schemas.
- Explain dependency injection.
- Manage DB sessions safely.
- Separate authentication and authorization.
- Explain JWT limitations and password hashing.
- Configure CORS intentionally.
- Keep secrets outside source control.
- Validate uploads and errors.
- Write unit/integration tests.

Reference notes

Primary documentation used for version-sensitive guidance. Verify exact package/API behavior against the versions installed in your project.

https://fastapi.tiangolo.com/

https://fastapi.tiangolo.com/tutorial/dependencies/

https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/

https://docs.pydantic.dev/

https://docs.sqlalchemy.org/

End of knowledge base document